

# MLOps Demo 02 — Lab Summary

Practical quality, validation and EC2 application-environment exercises completed hands-on

## Demo 02 — What We Achieved

Demo 02 focused on improving reliability around ML/software workflows through automated testing, data/code validation, and application execution/exposure on an AWS EC2 instance.

| Lab    | Main focus               | Key outcome                                                                    |
|--------|--------------------------|--------------------------------------------------------------------------------|
| Lab 01 | Unit testing with pytest | 2 automated tests passed; intentional code change demonstrated a failing test  |
| Lab 02 | Code/data validation     | Invalid data was rejected with a clear ValueError                              |
| Lab 03 | EC2 application exposure | Port 5000 access was configured and connectivity troubleshooting was performed |

## Lab 01 — Unit Testing with pytest

**Objective:** Automatically verify preprocessing behaviour instead of relying on manual checks.

**Implementation:** Tests covered `normalize_column`, including normalisation behaviour and the expected `ValueError` for a constant column.

**Issue encountered:** pytest initially failed with **ModuleNotFoundError: No module named 'src'** because the command was being run from the wrong nested project location.

**Fix:** Moved to the correct project directory and set `PYTHONPATH=$PWD`.

**Successful result:** 2 tests passed — `test_normalize_range` and `test_constant_column_raises`.

**Failure demonstration:** After changing preprocessing logic, the constant-column test failed with **Failed: DID NOT RAISE ValueError**. This proved that the test suite catches regressions.

**Learning:** Unit testing provides an automated quality gate and gives fast feedback when implementation changes break expected behaviour.

## Lab 02 — Code/Data Validation

**Objective:** Validate incoming data before it proceeds into downstream ML/data-processing steps.

**Validation rule demonstrated:** Age cannot be negative.

**Successful result:** The validation test suite completed with **2 passed**.

**Negative test:** A DataFrame containing `age = -5` was supplied deliberately. The validator rejected it with **ValueError: Age cannot be negative**.

**Learning:** Data validation is an early quality gate that prevents invalid or impossible values from reaching training or downstream processing.

## Lab 03 — EC2 Application / Port 5000 Exposure

**Objective:** Run an application/service on EC2 and make it reachable through the configured network path.

**AWS configuration:** The EC2 Security Group included TCP port **5000**, along with SSH, HTTP and HTTPS rules.

**Issue observed:** Opening **http://<EC2-public-IP>:5000** from the browser resulted in a connection timeout.

**Troubleshooting approach:** Check whether the application process is running, verify a listener with **ss -ltp | grep 5000**, test locally using **curl http://127.0.0.1:5000**, verify the EC2 public IP, and confirm the Security Group source rule.

**Learning:** External application access depends on the application process, listening address/port, host networking, AWS Security Group and correct public IP path all being aligned.

## Demo 02 — Interview Explanation

**30-second explanation:** “In Demo 02, I focused on software and data quality gates around an ML workflow. First, I implemented unit tests using pytest and verified both successful and failure scenarios. I then added data validation to reject invalid inputs such as negative age values. Finally, I worked with an EC2-hosted application and validated its external connectivity through port 5000, troubleshooting the application listener and AWS Security Group path. The overall goal was to make quality checks and failures visible early.”

### Key Commands / Evidence

- `pytest -v`
- `export PYTHONPATH=$PWD`
- python validation test with a deliberately invalid DataFrame
- `sudo ss -ltnp | grep 5000`
- `curl -v http://127.0.0.1:5000`
- AWS Security Group — TCP 5000 inbound rule

### Demo 02 Takeaways

- Automated tests catch regressions early.
- Correct project structure/PYTHONPATH is important for reliable test execution.
- Validation rules prevent bad data from entering downstream stages.
- Negative tests prove that invalid input is actively rejected.
- EC2 service availability requires both a healthy listener and correct network exposure.
- Troubleshooting is part of MLOps: use test output, process state, local connectivity and cloud networking evidence.

Prepared from the Demo 02 hands-on work completed in this session.